

USB Device Reference

The descriptors PhytoSense One presents,
the frames that cross the bus,
and what the host makes of them

Bretagne Namasté — PhytoScope 1.5.1 — 2026-09-23
Written for this project. MIT licensed.

Every byte here comes from **the firmware source**, not from a datasheet and not from memory. The normative documents are listed in section 14; we do not redistribute them, and where this document and they disagree, they are right.

Contents

1 — Scope, and what this document is not	3
2 — The physical layer, and why full speed	4
3 — The device descriptor, byte by byte	5
4 — The configuration, and the four interfaces	7
5 — Interfaces 0 and 1: the measurement stream	9
6 — The isochronous endpoint, and what a frame contains	11
7 — Interfaces 2 and 3: the control port	13
8 — The control protocol carried over CDC	14
9 — String descriptors, and the serial number	16
10 — Enumeration, transfer by transfer	17
11 — Known divergences from the descriptors	18
12 — What each host does with the device	21
13 — Diagnosing, and the failures that cost time	22
14 — Normative references	24

1 — Scope, and what this document is not

This is the reference for the USB side of **PhytoSense One**: the descriptors the board presents at plug-in, the two data paths it opens, the byte layout of what travels on each, and the way three operating systems bind drivers to it. It is written for whoever has to make a host talk to the board — someone writing an acquisition client, packaging a driver rule, or trying to work out why a freshly flashed board enumerates but produces nothing.

It is **not** a USB tutorial and not a specification. It assumes the reader knows what a descriptor and an endpoint are. Where a field is standard and unremarkable, it is named and left alone; where the board's choice is unusual or costly, the reasoning is given, because that is the part a specification cannot supply.

Nor is it a promise. Sections 3 to 9 describe what the firmware in this repository does at version 1.0.0. Section 11 records three places where the device's behaviour did *not* match what its own descriptors advertised; all three were settled on 2026-09-23, and they are kept because a board flashed with an earlier firmware still shows them, and because the reasoning outlives the defect. A reference that hides a divergence costs its reader more than one that has none to hide.

WHERE THE NUMBERS COME FROM

Constants and descriptor fields were read from `src/firmware/usb_descriptors.c` and `src/firmware/tusb_config.h`. Descriptor *lengths* are computed from TinyUSB's own macros by the C preprocessor, not added up by hand — the audio function's length in particular is the sum of thirteen terms, and adding those by hand is how documents come to disagree with their firmware.

Conventions

Multi-byte fields are little-endian, as USB requires: a `wTotalLength` of 219 appears on the wire as `DB 00`. Endpoint addresses are given as the host sees them, with bit 7 set for IN (device to host): `0x81` is IN endpoint 1. Byte values are written `0x`-prefixed; decimal values are bare.

2 — The physical layer, and why full speed

The board is a USB 2.0 **full-speed** device: 12 Mbit/s, one frame per millisecond, over a USB-C receptacle wired for USB 2.0 only. It is bus-powered. The RP2350's USB controller has no high-speed PHY, so full speed is not a choice so much as the ceiling; what follows is why that ceiling is comfortably above the need.

The bandwidth actually required

Four channels, three bytes per sample, 250 samples per second:

```
4 channels x 3 bytes x 250 Hz = 3000 bytes/s = 3 kB/s
```

Full speed offers roughly 1023 bytes per frame to a single isochronous endpoint, which is 1023 kB/s. The measurement stream uses three parts in a thousand of it. Even the 32 kHz mode the analog front end can be driven at comes to 384 kB/s, still inside the budget with the control port alongside.

WHY THIS MATTERS MORE THAN IT LOOKS

An instrument whose bus is nearly full has no margin for a host that schedules badly, and isochronous transfers are *not retried*: a frame the host fails to collect is gone. Running at three parts in a thousand means that a lost frame is always the host's doing and never a bandwidth shortage — which turns a whole class of intermittent faults into a diagnosable one. The counter in section 8's beacon exists to say so.

Cabling, and the cable that is not a cable

The receptacle is USB-C, but only the USB 2.0 pairs are connected: there is no SuperSpeed wiring, no alternate mode and no Power Delivery negotiation. Any USB-C cable that carries data will work, in either orientation.

The failure that costs the most time is a cable that carries power but not data — common among cables shipped with chargers. The board then powers up, its indicator behaves normally, and the host sees nothing at all. A board that has power and does not enumerate is almost always this.

Current

Quantity	Value	Where it comes from
Declared in the configuration descriptor	500 mA	TUD_CONFIG_DESCRIPTOR(..., 500), encoded as bMaxPower = 250 in 2 mA units
Typical draw, streaming	~180 mA	Measured on the bench; see the board fascicle's power budget
Margin	~320 mA	Covers the inrush when the isolated supplies come up

Declaring 500 mA when 180 mA is used is deliberate. The isolated rails draw a brief surge at start-up, and a hub that has granted only 100 mA may cut the port during it. Asking for the full bus-powered allowance costs nothing on a host with a normal budget and avoids an enumeration that succeeds and then drops.

3 — The device descriptor, byte by byte

Eighteen bytes, returned for the first `GET_DESCRIPTOR(Device)`. This is the whole of it, with the value each field actually carries:

Offset	Field	Value	Meaning
0	bLength	0x12	18 bytes
1	bDescriptorType	0x01	DEVICE
2	bcdUSB	0x0200	USB 2.0 (at full speed)
4	bDeviceClass	0xEF	Miscellaneous
5	bDeviceSubClass	0x02	Common class
6	bDeviceProtocol	0x01	Interface Association Descriptor
7	bMaxPacketSize0	0x40	64 bytes on endpoint 0
8	idVendor	0x1209	pid.codes, the allocation for open projects
10	idProduct	0x7A01	PhytoSense One
12	bcdDevice	0x0100	Firmware 1.0.0
14	iManufacturer	0x01	String 1, "Bretagne Namaste"
15	iProduct	0x02	String 2, "PhytoSense One"
16	iSerialNumber	0x03	String 3, built at run time (section 9)
17	bNumConfigurations	0x01	One configuration

On the wire, in order:

```
12 01 00 02 EF 02 01 40 09 12 01 7A 00 01 01 02 03 01
```

The three bytes that decide whether Windows works

`bDeviceClass`, `bDeviceSubClass` and `bDeviceProtocol` are `0xEF`, `0x02`, `0x01`. That triple is the standard signature of a **composite device described by Interface Association Descriptors**. It says: do not look for a class here, look at each interface, and honour the IADs that group them.

Getting it wrong is not a cosmetic error. With any other triple, Windows treats the device as single-function, binds one driver to the first interface it recognises, and never reaches the CDC interfaces — so the board appears as an audio device with no control port, and no amount of driver-side effort fixes it because the host has already decided. Linux is more forgiving and will bind per-interface anyway, which is precisely why this class of bug survives testing on Linux and is found on Windows.

A TRAP FOR ANYONE FORKING THIS FIRMWARE

It is tempting to set `bDeviceClass` to the audio class, since audio is the board's main function. Do not. The moment a second function exists, the device is composite, and the only portable way to say so is the Miscellaneous/Common/IAD triple above.

The vendor and product identifiers

`0x1209` is the **pid.codes** vendor identifier, an allocation shared by open-hardware projects, and `0x7A01` is this board within it. The pair is what every udev rule, every driver-matching table and every diagnostic in this project keys on; it is stable across firmware versions, whereas `bcdDevice` is not.

Anyone rebuilding this firmware for a different board should change the product identifier. Two different boards answering to the same VID:PID will be matched by the same rules, and the first diagnostic to run on a machine with both attached will report whichever it found first.

4 — The configuration, and the four interfaces

There is one configuration, holding four interfaces in two functions. Its total length is **219 bytes**, made of three parts:

Part	Bytes	TinyUSB macro
Configuration header	9	TUD_CONFIG_DESC_LEN
Audio function (UAC2, four-channel input)	144	TUD_AUDIO_MIC_FOUR_CH_DESC_LEN
CDC function (virtual serial port)	66	TUD_CDC_DESC_LEN
Total	219	wTotalLength , on the wire DB 00

The configuration descriptor

Offset	Field	Value	Meaning
0	bLength	0x09	9 bytes
1	bDescriptorType	0x02	CONFIGURATION
2	wTotalLength	0x00DB	219 bytes, this descriptor included
4	bNumInterfaces	0x04	Four
5	bConfigurationValue	0x01	Selected by SET_CONFIGURATION(1)
6	iConfiguration	0x00	No string
7	bmAttributes	0xA0	Bus-powered, remote wake-up supported
8	bMaxPower	0xFA	250 x 2 mA = 500 mA

The interfaces, and the order they appear in

#	Interface	Class	Endpoints
0	Audio Control	Audio (0x01), subclass AUDIOCONTROL	none
1	Audio Streaming	Audio (0x01), subclass AUDIOSTREAMING	0x81 isochronous IN, on alternate setting 1
2	CDC Communication	CDC (0x02), Abstract Control Model	0x82 interrupt IN, 8 bytes
3	CDC Data	CDC Data (0x0A)	0x03 bulk OUT and 0x83 bulk IN, 64 bytes

Interfaces 0 and 1 are grouped by one IAD, interfaces 2 and 3 by another. The IAD is what tells a composite-aware host that interface 1 belongs with interface 0 and must not be handed to a different driver.

The audio function comes **first**, and that is deliberate. Some older hosts stop exploring a composite device at the first function whose class they can handle; putting the measurement stream first means that on such a host the instrument still measures, losing only the control port. The reverse order would give a host that can be configured but cannot acquire, which is the less useful half.

Endpoint map

Address	Direction	Type	Max packet	Carries
0x00 / 0x80	both	Control	64	Enumeration, class requests
0x81	IN	Isochronous	384	The measurement stream
0x82	IN	Interrupt	8	CDC serial-state notifications
0x03	OUT	Bulk	64	Commands to the board
0x83	IN	Bulk	64	Replies and beacons

5 — Interfaces 0 and 1: the measurement stream

The measurement path is declared as a **USB Audio Class 2.0 input** — in the class's own vocabulary, a four-channel microphone. It is built from TinyUSB's `TUD_AUDIO_MIC_FOUR_CH_DESCRIPTOR` template with three parameters supplied by this firmware:

Parameter	Value	Effect
<code>_nBytesPerSample</code>	3	Three bytes on the wire per sample
<code>_nBitsUsedPerSample</code>	24	All 24 bits carry data
<code>_epin</code>	0x81	The isochronous IN endpoint
<code>CFG_TUD_AUDIO_FUNC_1_N_CHANNELS_TX</code>	4	Four channels
<code>CFG_TUD_AUDIO_EP_SZ_IN</code>	384	Maximum packet size — 32 x 12, checked by a static assert

Why audio class, for an electrometer

Nothing about a plant's membrane potential is audio. The class was chosen for what it gives the user, not for what it describes:

- **No driver to install.** UAC2 is supported out of the box by Linux, Windows 10 and later, and macOS. A vendor-class endpoint would mean shipping a driver, signing it, and maintaining it for three operating systems — against constraint C-40, which forbids making the user install anything mandatory.
- **Isochronous scheduling for free.** The class brings a guaranteed slot in every frame and a host-side buffering model built for a continuous stream with a fixed cadence, which is exactly the traffic shape.
- **Existing tools.** Anything that can record from a sound card can record the raw stream, which makes the board inspectable with software the user already has.

AND WHAT IT COSTS

The price is that the host's audio stack believes it is handling audio. It may offer to resample, to apply gain, or to mix — every one of which would destroy the measurement. Section 6 explains how the asynchronous clock declaration keeps the host from resampling, and section 11 records where this choice is not yet fully honoured.

Interface 0: audio control

Interface 0 carries no endpoint. It exists to hold the function's topology, which UAC2 expresses as a graph of entities:

Clock Source, ID 0x04

Declared `AUDIO_CLOCK_SOURCE_ATT_INT_FIX_CLK` — an internal, fixed clock — with its frequency marked *readable*. That last attribute has a consequence the firmware does not yet meet; see section 11.

Input Terminal, ID 0x01

Type `AUDIO_TERM_TYPE_IN_GENERIC_MIC`, four channels, associated with the output terminal. This is the entity the four electrode inputs are modelled as.

Feature Unit, four channels

Declared with mute and volume controls, because the template declares them. They are not wired to anything in the analog front end: gain is set through the control port (section 8), not through this unit.

Output Terminal, ID 0x03

Type USB streaming — the point at which the samples leave for the host.

Interface 1: audio streaming, and its two alternate settings

Interface 1 has two alternate settings, which is how UAC2 idles:

Alt	Endpoints	State
0	none	Idle. No bus bandwidth is reserved.
1	0x81 , isochronous IN, 384 bytes	Streaming.

A host that is not recording leaves the interface on alternate setting 0, and the board's isochronous slot costs the bus nothing. Streaming begins when the host issues `SET_INTERFACE(1, 1)`, and the firmware's `tud_audio_tx_done_pre_load_cb` starts being called once per frame.

WHAT THIS MEANS FOR A CLIENT

Opening the control port does not start the stream, and closing it does not stop it: they are separate functions with separate state. A client that wants samples must select alternate setting 1 — which, on every host, means opening the capture device rather than sending a command.

6 — The isochronous endpoint, and what a frame contains

Endpoint 0x81 carries the measurement. Its declared maximum packet size is **384 bytes** — 32 sample sets exactly, the fastest the analog front end can be driven — its interval is one frame (1 ms at full speed), and it has **no feedback endpoint**. It was 256 bytes before 2026-09-23; see 11.2.

Sample layout

A sample set is four channels of three bytes, little-endian, two's complement, 24 bits significant: twelve bytes, channel 0 first.

```
offset  0  1  2   3  4  5   6  7  8   9 10 11
        <--ch0--> <--ch1--> <--ch2--> <--ch3-->
        LSB..MSB  LSB..MSB  LSB..MSB  LSB..MSB
```

The firmware writes each sample as three separate bytes, least significant first, which is what the host's audio stack expects of a 24-bit little-endian format:

```
trame_usb[n++] = (uint8_t)(code & 0xFF);
trame_usb[n++] = (uint8_t)((code >> 8) & 0xFF);
trame_usb[n++] = (uint8_t)((code >> 16) & 0xFF);
```

Frame length is variable, and this surprises people

The endpoint's packet is *not* a fixed twelve bytes. Once per frame the firmware drains whatever the acquisition ring holds, up to the packet size:

```
while (n + (AFE_VOIES * 3u) <= sizeof(trame_usb) && tampon_tirer(&e)) {
    /* ... three bytes per channel ... */
}
tud_audio_write(trame_usb, (uint16_t)n);
```

At the default 250 Hz, a sample set becomes available every 4 ms, so the steady state is a repeating pattern of three empty frames and one carrying twelve bytes. At 1 kHz it is one sample set per frame. At 32 kHz it is 32 sets, 384 bytes — which is exactly the declared maximum, and the reason the maximum is that number rather than a round one.

Rate	Sets per frame	Bytes per frame	Average	Fits in 384 B?
250 Hz (default)	0 or 1	0 or 12	3 B/ms	yes
1 kHz	1	12	12 B/ms	yes
8 kHz	8	96	96 B/ms	yes
16 kHz	16	192	192 B/ms	yes
32 kHz	32	384	384 B/ms	yes — exactly, with nothing to spare

A CLIENT MUST NOT ASSUME A FRAME LENGTH

A reader that expects twelve bytes per frame will work at 1 kHz and fail at every other rate. Read whatever length the transfer returns and treat the stream as a byte sequence in which every twelve bytes is one sample set — which is what a host audio stack does anyway, and why recording through the sound API works when a hand-rolled reader does not.

Asynchronous clock, and no feedback

Two settings in `tusb_config.h` decide the timing model:

```
#define CFG_TUD_AUDIO_ENABLE_FEEDBACK_EP    0
#define CFG_TUD_AUDIO_FUNC_1_CLK_SRC_ASYNC  1
```

Asynchronous means **the board sets the cadence, not the host**. The sample clock is the board's own, derived from its oscillator; the host adapts. This is the only defensible choice for an instrument: if the host supplied the clock, or if the board resampled to follow one, the time base of every recorded measurement would be the host's scheduler rather than a crystal, and a timestamp would stop meaning anything.

The absence of a feedback endpoint is the consequence. A feedback endpoint exists so the host can be told to speed up or slow down its delivery — which only makes sense for output. For an asynchronous input the host simply takes what arrives, and the stream carries no rate correction at all.

HOW DRIFT SHOWS UP, AND WHAT TO DO ABOUT IT

Board and host clocks differ by some tens of parts per million. Over an hour that is tens of milliseconds of accumulated offset — invisible within a recording, and real between a recording and an external event. The board therefore ships its own sample counter: the `clock` command (section 8) returns `n`, the count since power-up, and `t_s`, that count divided by the sample rate. Timestamp against `n`, never against the host's clock, and the drift becomes a constant offset that can be measured once instead of an error that grows.

Lost samples are counted, not hidden

The ring buffer between the acquisition loop and the USB callback has a fixed size. When the host does not collect frames fast enough, the producer overruns the consumer, and the firmware increments a counter rather than blocking:

```
if (/* ring full */) {
    echantillons_perdus++;    /* the host is not reading fast enough */
}
```

That counter is reported in every beacon as `lost`, and by the `clock` command. A recording whose `lost` is non-zero has gaps, and the number says how many. This is worth checking at the end of every session: isochronous transfers are not retried, so nothing else will tell you.

7 — Interfaces 2 and 3: the control port

The second function is a **CDC-ACM virtual serial port**: interface 2 for control, interface 3 for data, grouped by their own IAD.

Endpoint	Type	Size	Use
0x82	Interrupt IN	8	Serial-state notifications (<code>SERIAL_STATE</code>)
0x03	Bulk OUT	64	Command lines from the host
0x83	Bulk IN	64	Replies, and unsolicited beacons

Buffering, from `tusb_config.h`:

```
#define CFG_TUD_CDC_RX_BUFSIZE 256
#define CFG_TUD_CDC_TX_BUFSIZE 512
```

Those sizes match the protocol's own limits — `PROTO_LIGNE_MAX` is 256 and `PROTO_REPONSE_MAX` is 512 — so a maximum-length command or reply never needs to be split across buffer refills. The asymmetry is because replies are JSON and longer than the commands that ask for them: the answer to `selftest` runs to several hundred bytes, while the command is eight.

Why a serial port rather than control transfers

Configuration could have gone over endpoint 0 as vendor requests. A serial port was chosen because it is **reachable by hand**. Anyone with `screen`, `minicom` or PuTTY can type `selftest` and read the answer, with no client, no library and no tooling — which matters for a board meant to be built by its user. A vendor control interface would make the instrument opaque to everyone but the author of its client.

BAUD RATE IS A FICTION

CDC-ACM carries a baud rate because it emulates a UART, and the board ignores it entirely: the data travels in bulk transfers at bus speed. Any setting works. Line discipline, on the other hand, is not a fiction — see section 13 for what a terminal's local echo and line ending do to this protocol.

8 — The control protocol carried over CDC

The protocol is deliberately plain: **one command per line, one reply per line**. A line is CR-LF terminated, at most 256 bytes in and 512 bytes out. Every reply begins with a single character that says how to read the rest.

First byte	Meaning	Body
+	Success	A JSON object
-	Failure	A JSON object with a <code>message</code> field

A one-character status ahead of the JSON means a client can branch before parsing, and a human reading the port can see at a glance whether something worked. The body is JSON so that a client need not write a parser.

The command set

Com-mand	Argument	Returns
? or id	none	Model, serial, firmware version, channel count, sample rate, front-end type and serial, the gain table, current gain and range, volts per unit
gain	2, 10, 20, 200 or auto	The gain applied, and whether auto-ranging is on. The argument is the gain itself, not an index into the table
range	0 .. 3	The selected range and the reference resistance in ohms
offset	a DAC code	The offset applied
clock	none	n (samples since power-up), t_s (n / sample rate), tcxo_ppm, lost
mark	a label, up to 48 printable characters	The sample index at which the mark was placed, and the sanitised label
selftest	none	Pass or fail; the three reference resistors as measured (1 M, 10 M, 100 M); the error on each; the noise floor in volts; the leakage in femtoamperes
env	none	Temperature in degrees Celsius; humidity, pressure and illuminance, reserved
save	none	Commits the current settings to flash
help	none	The command names, as a JSON array

An exchange, as it appears on the port

```
> ?
+{"model":"PhytoSense One","serial":"PS1-4A17C302","firmware":"1.0.0",
  "channels":4,"sample_rate":250.0,"frontend":"ADA4530-1",
  "frontend_serial":"...", "gains":[2,10,20,200], "gain":10, "range":1,
  "volts_per_unit":1.0}

> gain 200
+{"gain":200, "auto":false, "message":"gain forced"}

> gain 7
-{"message":"gain not in table: 2, 10, 20 or 200"}

> clock
+{"n":1250000, "t_s":5000.000, "tcxo_ppm":-0.4, "lost":0}

> mark stimulus-3
+{"n":1250412, "label":"stimulus-3"}
```

THE GAIN ARGUMENT IS THE GAIN, NOT AN INDEX

`gain 200` asks for a gain of 200, and the firmware searches the table for it. `gain 3` is an error, not "the fourth entry". This reads better at a terminal and is the one place where the protocol chose the human over the program.

Unsolicited output: the beacon

The board writes a beacon to the port on its own, without being asked:

```
+{"beacon":1,"n":1250000,"lost":0,"gain":10,"range":1,"offset":2048,"sat":false}
```

It carries the sample index, the lost-sample count, the current gain, range and offset, and whether the front end is saturated. Its purpose is to let a client follow the instrument's state — and in particular notice saturation and loss — without polling.

A CLIENT MUST TOLERATE AN UNREQUESTED LINE

A reader that sends a command and takes the next line as its answer will eventually read a beacon instead, and mis-attribute it. Beacons are identifiable: their object carries `"beacon":1`, which no reply does. Filter on that, and read on.

Beacons are only written when the port is open — `tud_cdc_connected()` is checked first — so an instrument nobody is listening to does not fill a buffer with them.

Marks, and why the label is filtered

`mark` exists to tie an external event to a sample index: the operator touches the plant, types `mark touch`, and the recording carries the exact sample at which it happened. The label is copied through a filter that keeps only printable ASCII and drops the double quote and backslash:

```
if (*p >= 32 && *p < 127 && *p != '"' && *p != '\\') {
    propre[k++] = *p;
}
```

The reply is JSON, and an unescaped quote in a label would produce a document the client cannot parse — from a string the operator typed. Filtering at the source is cheaper than escaping correctly in an embedded `sprintf`, and it cannot be got wrong later. An empty label becomes a default rather than an empty string.

9 — String descriptors, and the serial number

Index	Content	Referenced by
0	Language table: 0x0409 , English (United States)	The host, first
1	Bretagne Namaste	iManufacturer
2	PhytoSense One	iProduct
3	PS1-XXXXXXX , built at run time	iSerialNumber
4	PhytoSense Measurement Input	The audio function
5	PhytoSense Control	The CDC function

The strings are stored as ASCII and widened to UTF-16LE on demand, into a 32-word buffer. A string longer than 31 characters is **truncated rather than allowed to overflow** — none of the strings above comes close, but the guard is there for whoever adds one:

```
if (n > TU_ARRAY_SIZE(desc_str) - 1) {
    n = TU_ARRAY_SIZE(desc_str) - 1;
}
```

The manufacturer string is written "Bretagne Namaste", without the acute accent the name carries elsewhere in this project. The widening loop copies each byte into a 16-bit word, which is correct for ASCII and wrong for any multi-byte UTF-8 sequence: an "é" would arrive as two spurious characters. Unaccented here is a deliberate consequence of a cheap conversion, not an oversight.

The serial number

Built once, on the first request, from the RP2350's unique flash identifier:

```
pico_unique_board_id_t uid;
pico_get_unique_board_id(&uid);
/* the last four bytes, in uppercase hexadecimal */
```

The form is PS1- followed by eight hexadecimal digits: the low 32 bits of the 64-bit identifier. PS1-4A17C302 is a representative value.

Thirty-two bits rather than sixty-four is a compromise stated plainly in the firmware's own comment: enough to be unique across any plausible production run, short enough to be copied onto a lab notebook by hand. Both halves of that matter. A serial nobody transcribes is a serial that never appears in the record of an experiment.

WHY A REAL SERIAL NUMBER AT ALL

Because two boards will eventually be attached to one machine, and a recording has to say which one made it. The serial is in the USB descriptor, so the host knows it before any software runs; it is in the ? reply, so a client can record it; and it is stable across reflashes, because it comes from the flash chip rather than from the firmware image.

10 — Enumeration, transfer by transfer

What happens between plugging the board in and a client reading samples. Steps 1 to 7 are the standard sequence and are listed so that a trace can be compared against them; steps 8 and 9 are where this device's own behaviour begins.

1. **Reset and address.** The host resets the port and assigns an address. The board answers on endpoint 0 with 64-byte packets (`bMaxPacketSize0 = 0x40`).
2. **GET_DESCRIPTOR(Device)** . The eighteen bytes of section 3. The host now knows the device is composite with IADs, and that it must not bind a single driver.
3. **GET_DESCRIPTOR(Configuration), nine bytes.** The host reads the header to learn `wTotalLength = 219`.
4. **GET_DESCRIPTOR(Configuration), 219 bytes.** The whole tree: two IADs, four interfaces, the UAC2 entity graph, the CDC functional descriptors, five endpoint descriptors.
5. **String descriptors.** Index 0 for the language table, then 1, 2 and 3. Index 3 triggers the one-time construction of the serial.
6. **SET_CONFIGURATION(1)** . The device is configured. Interface 1 is on alternate setting 0, so no isochronous bandwidth is reserved yet.
7. **Driver binding.** The host attaches its audio class driver to interfaces 0 and 1, and its CDC-ACM driver to interfaces 2 and 3. The control port appears as a device node (section 12).
8. **The audio driver interrogates the clock.** Having seen a clock source whose frequency is marked readable, the host issues a class-specific `GET CUR` on `CS_SAM_FREQ_CONTROL` . **The board stalls it** — see section 11. What follows depends on the host's tolerance.
9. **SET_INTERFACE(1, 1)** . When the host does open the stream, this selects alternate setting 1, endpoint `0x81` is scheduled, and `tud_audio_tx_done_pre_load_cb` begins to be called once per frame.

READING A TRACE

On Linux, `usbmon` shows all of the above. The transfer worth finding is step 8: a `GET CUR` on the clock entity answered with a `STALL`. Its presence confirms that the host asked and the board refused, which is the difference between "the stream did not open" and "the stream opened and produced nothing".

11 — Known divergences from the descriptors

Three places where the device did not do what its own descriptors advertised. **All three were settled on 2026-09-23**, and they are kept here rather than deleted: a reader holding a board flashed with an earlier firmware needs to know what it will meet, and the reasoning is worth more than the tidiness.

11.1 — The clock frequency: settled on 2026-09-23

The UAC2 clock source (entity `0x04`) is declared with a read control on its sample-frequency attribute. A host is therefore entitled — and in practice does — to issue a class-specific `GET CUR` on `CS_SAM_FREQ_CONTROL`.

Answering that request is the job of `tud_audio_get_req_entity_cb`. Until 2026-09-23 this firmware did not define it, and TinyUSB's weak default — a log line and `return false` — stalled a request the board had itself invited. A permissive host carried on with the format it had read from the descriptors; a strict one refused to open the stream. The symptom was a device that enumerated perfectly, appeared as a four-channel capture device, and produced silence or an open error, with nothing in any log to connect it to a control transfer.

The firmware now answers, and it answers the truth. `usb_descriptors.c` defines both entity callbacks:

Entity	Request	Answer
0x04 clock source	GET CUR CS_SAM_FREQ	250, as a 4-byte CUR block
	GET RANGE CS_SAM_FREQ	one sub-range, bMin = bMax = 250, bRes = 0
	SET CUR CS_SAM_FREQ	accepted for 250, stalled for anything else
0x04 clock source	GET CUR CS_CLK_VALID	1 — it is a crystal
0x02 feature unit	GET CUR mute / volume	0 (not muted) / 0 (0 dB)
	GET RANGE volume	one sub-range, all zero — the slider cannot move
0x01 input terminal	GET CUR CONNECTOR	4 channels, no predefined spatial position

A single sub-range with `bRes = 0` is how UAC2 says "this value is fixed", which agrees with `AUDIO_CLOCK_SOURCE_ATT_INT_FIX_CLK` in the descriptor: the rate is fixed by the analog chain — 12.288 MHz / 2 / 4096 / 6 — and nothing on the board can change it.

WHY 250 AND NOT A RATE A HOST WOULD LIKE

Most host audio stacks will not accept a sample rate below 8 kHz, so a host may refuse the stream on reading this answer. **That refusal is the correct outcome.** The alternative is to report a rate the converter does not run at — 8 kHz, say — and then every timestamp the host derives from it is wrong, silently, for as long as the recording lasts. A host that refuses tells you at plug-in; a host misled by a convenient lie tells you months later, if ever, when two recordings of the same plant disagree about when something happened.

This is why the board carries a *second* path: the measurement stream is also fully described by the control port (section 8), which owes nothing to the audio class and its rate conventions. A client that reads the board through `libusb` or through the CDC port is not affected by any of this.

The one thing still worth doing here is in the descriptor rather than in the callback: the feature unit's mute and volume are declared *read/write* because TinyUSB's four-channel microphone template declares them so, and they are wired to nothing (11.3). Declaring them read-only would mean writing the audio descriptor out by hand instead of using the template. Recorded, not done.

11.2 — The endpoint size: settled on 2026-09-23

`tusb_config.h` sized the isochronous packet at 256 bytes, with a comment claiming it absorbed "the 1 kHz and 32 kHz modes without reconfiguring the stack". It did not. The drain loop in `main.c` fills a frame with *whole* sample sets and stops when the next one would not fit, so the packet size is a hard ceiling on the rate:

```
highest rate = packet size / 12 bytes x 1000 frames/s
```

256 bytes gives 21 sample sets — 252 bytes used, four wasted in every frame — and a ceiling near 21 kHz. Above it the loop left samples in the ring, where the `lost` counter recorded them; a claim about a mode nobody could select, which is how it went unnoticed.

The packet is now 384 bytes, which is 32 x 12 exactly: the 32 kHz mode with nothing wasted. A `TU_VERIFY_STATIC` in `main.c` makes a packet size that is not a whole number of sample sets a build error, so the next change to it cannot quietly reintroduce the waste.

WHAT THE LARGER PACKET COSTS

A host reserves 384 bytes of every frame from the moment it selects alternate setting 1, used or not — 384 kB/s of bus held for a stream that currently carries 3 kB/s. Full speed allows a single isochronous endpoint up to 1023 bytes per frame, so this asks for 37 % of what one endpoint may claim, on a bus otherwise running at three parts in a thousand of its capacity. Alternate setting 0 reserves nothing, and that is what the board sits in whenever nobody is capturing.

That is the price of a descriptor that does not lie. The alternative — 256 bytes and a comment about a mode that does not work — costs nothing until somebody believes it.

11.3 — The phantom mute and volume: settled on 2026-09-23

The audio control interface declared a four-channel feature unit with mute and volume controls, on the master and on all four channels, because TinyUSB's microphone template declares them. Nothing on the board is wired to them: the gain is set through the control port, in decade steps chosen by a relay tree (section 8). A host offered a volume slider that did nothing — and worse, an application could set it, read it back unchanged, and draw its own conclusions.

The first attempt, the same morning, was to *answer* those requests with fixed values — not muted, 0 dB, a range allowing nothing else. That is a polite fiction, and it was replaced the same day by the honest fix: **the audio function is now written out by hand** in `usb_descriptors.c`, instead of taken from the template, and the feature unit declares `AUDIO_CTRL_NONE` everywhere. A host shows no slider at all, which is the truth, and a request about a control the device never advertised is stalled — which is the correct answer to it.

The entity graph is unchanged, deliberately: `0x01`, `0x02`, `0x03` and `0x04` are what a host addresses in a class request and what section 5 documents, and the configuration descriptor is still 219 bytes. What changed is four `bmaControls` fields, from `0x03` to `0x00`.

WHY GAIN IS NOT A VOLUME CONTROL

It would have been easy to wire the volume control to the gain and call the divergence fixed. It would also have been wrong: gain changes the measurement's **full scale**, and a host that moved it without recording the change would invalidate every sample that followed. The control port's `gain` command exists because the change has to be written into the session's metadata, next to the samples it applies to. A mixer slider has nowhere to write that.

12 — What each host does with the device

Linux

Interfaces	Driver	Appears as
0, 1	snd-usb-audio	An ALSA capture device, four channels, S24_3LE
2, 3	cdc_acm	/dev/ttyACMn

Both drivers are in-tree, so nothing needs installing. Access to `/dev/ttyACMn` normally requires membership of the `dialout` group, which is the commonest reason a client that works under `sudo` fails without it. A udev rule keyed on the VID:PID avoids both the group and the privilege:

```
# /etc/udev/rules.d/70-phytosense.rules
SUBSYSTEM=="tty", ATTRS{idVendor}=="1209", ATTRS{idProduct}=="7a01", \
    MODE="0660", TAG+="uaccess", SYMLINK+="phytosense-%s{serial}"
```

The symlink carries the serial, so two boards on one machine get stable, distinguishable names rather than `ttyACM0` and `ttyACM1` in whichever order they were found.

Windows

Interfaces	Driver	Appears as
0, 1	usbaudio2.sys	A recording device in Sound settings
2, 3	usbser.sys	COMn

Windows 10 version 1703 and later bind both without a driver package, and both depend on the IAD signature of section 3 being exactly right. On Windows 7 and 8 the CDC interface needs an `.inf` to point `usbser.sys` at it; those versions are out of support and out of scope here.

macOS

Interfaces	Driver	Appears as
0, 1	AppleUSBAudio	A Core Audio input device
2, 3	AppleUSBBCDACM	/dev/cu.usbmodemnnnn

Use `/dev/cu.*` and not `/dev/tty.*`. The `tty` node waits for carrier detect, which a CDC device does not assert, so opening it blocks; `cu` does not wait. This costs an hour the first time and is the single most common macOS problem with any CDC device.

ONE BEHAVIOUR ALL THREE SHARE

Opening the control port asserts DTR, and many terminal programs toggle it on connect. The board does not reset on DTR — unlike a great many microcontroller boards — so a connection does not interrupt a recording in progress. That is a property worth knowing before someone connects a terminal to a running experiment to see what it is doing.

13 — Diagnosing, and the failures that cost time

Confirming the device from outside

```
$ lsusb -d 1209:7a01
Bus 001 Device 014: ID 1209:7a01 pid.codes PhytoSense One

$ lsusb -v -d 1209:7a01 | head -20      # descriptors, field by field
$ lsusb -t                             # which driver bound to which interface
```

`lsusb -t` is the one to reach for first: it shows the interface tree with the driver attached to each. Four interfaces with `snd-usb-audio` on two and `cdc_acm` on two means the descriptors were understood. Anything else — one driver, or none — points at section 3.

The project ships its own check, which goes further than enumeration:

```
$ python3 -m phytoscope.core.usbdiag
$ phytoscope --check
```

The failures, in the order they are met

Symptom	Most likely cause	Test
Nothing in <code>lsusb</code> ; board has power	A charge-only USB-C cable	Try a cable known to carry data. This is the commonest fault by a wide margin
Enumerates, then disappears	Port current limit hit by the isolated rails' in-rush	A powered hub, or a rear port rather than a front one
Audio device present, control port absent (Windows)	IAD signature not honoured — the host bound one driver	<code>lsusb -t</code> on Linux for comparison; check the three bytes of section 3
Capture device present, always silent	Alternate setting 1 never selected; or, on firmware older than 2026-09-23, the clock request stalled	Open the capture device; then look for a stalled GET CUR in <code>usbmon</code> (section 11.1)
Opening the port blocks forever (macOS)	<code>/dev/tty.*</code> instead of <code>/dev/cu.*</code>	Open <code>cu</code>
Permission denied on <code>/dev/ttyACM0</code>	Not in <code>dialout</code> , and no udev rule	Install the rule in section 12
Commands ignored; the port is open	Line ending. The board wants CR-LF	In <code>screen</code> , press Ctrl-J after the line, or configure CR-LF
Replies arrive garbled or doubled	The terminal's local echo, mixed into the stream	Turn local echo off
A reply does not match the command sent	A beacon was read as the answer	Skip objects carrying <code>"beacon":1</code> (section 8)
<code>lost</code> is non-zero at the end of a recording	The host did not collect frames fast enough	Close other capture clients; check for a hub shared with a busy device
Two boards, the wrong one answers	Both match the same VID:PID	Use the serial-bearing symlink, or the <code>serial</code> field of ?

THE CHECK WORTH MAKING A HABIT

Read `clock` at the start and at the end of every session. Two numbers — the sample index and `lost` — tell you how many samples the recording should contain and how many went missing. A recording whose arithmetic does not close is one to repeat, and it is far cheaper to know that at the bench than during analysis.

14 — Normative references

None of these is redistributed with this project. Where this document and any of them disagree, they are right.

Universal Serial Bus Specification, Revision 2.0

USB Implementers Forum, 2000, with its later engineering change notices. Chapter 9 defines the descriptors of sections 3 and 4 and the enumeration of section 10.

USB Device Class Definition for Audio Devices, Release 2.0

USB-IF, 2006. Defines the entity graph of section 5, the alternate-setting idiom, the asynchronous clock model of section 6, and the `CS_SAM_FREQ_CONTROL` request of section 11.1.

USB Device Class Definition for Audio Data Formats, Release 2.0

USB-IF, 2006. Defines the Type I PCM format and the three-byte, 24-bit-significant layout of section 6.

USB Class Definitions for Communication Devices, Revision 1.2

USB-IF, 2007, plus the PSTN subclass document. Defines the Abstract Control Model of section 7.

USB Interface Association Descriptor, Engineering Change Notice

USB-IF, 2005. The source of the `0xEF / 0x02 / 0x01` device triple of section 3.

RP2350 Datasheet

Raspberry Pi Ltd. The USB device controller, its endpoint budget, and the unique board identifier used for the serial number of section 9.

TinyUSB

The device stack this firmware is built on, vendored with the Pico SDK. `src/device/usbd.h` holds the descriptor templates whose lengths section 4 quotes and whose entity numbering section 11.1 answers to, and `src/class/audio/audio_device.c` the weak callback that section overrides.

pid.codes

The allocation under vendor identifier `0x1209` from which `0x7A01` is taken.

Within this project

`src/firmware/usb_descriptors.c`

Every descriptor in sections 3, 4, 5 and 9.

`src/firmware/tusb_config.h`

The stack configuration of sections 6 and 7.

`src/firmware/protocol.c`

The command set, reply format and beacon of section 8.

`src/firmware/main.c`

The acquisition ring, the frame-filling loop of section 6, and the lost-sample counter.

`src/phytoscope/phytoscope/core/usbdia.py`

The host-side check of section 13.

MIDI Protocol Reference, `sources/midi/`

The other protocol this instrument speaks, on the way out rather than in.